Internet Technology Lab Report

Assignment – 2

Problem Statement:

Implement a WebSocket-based Photo Sharing Application. The application must be an asynchronous multi-client system where users can share photos and comments publicly or within specific groups

---

Student Details

- **Name:** Sumesh Ranjan Majee
- **Roll Number:** 002310501020
- **Year:** 3rd Year B.C.S.E
- **Group:** A1
- **Date:** 11th February 2026

Subject:

Internet Technology

1. Design

1.1 Purpose

The purpose of this program is to demonstrate the implementation of a real-time, full-duplex communication protocol using WebSockets. Unlike traditional HTTP request-response models, this application establishes a persistent connection between the client and server to facilitate immediate broadcasting of multimedia data (photos) and text (comments) among concurrent users. It specifically explores the concepts of "broadcasting" (Public Feed) versus "multicasting" (Group/Room logic).

1.2 System Structure

The system follows a Client-Server architecture utilizing the WebSocket protocol.

1. **Server (Node.js):**
    - **HTTP Server:** Uses express to serve static assets (HTML, CSS, JS).
    - **WebSocket Server:** Uses the ws library to upgrade HTTP connections to WebSocket connections. It maintains state using two Hash Maps: clientRooms (tracking which user is in which room) and clientNames (mapping sockets to usernames).
2. **Client (Browser):**
    - **UI Layer:** HTML/CSS interface with distinct panels for "Public Feed" and "Group Zone".
    - **Logic Layer:** script.js handles DOM manipulation and WebSocket events. It converts images to Base64 strings using FileReader before transmission.

1.3 Input and Output Format

- **Input:**
    - *User Action:* A JSON object sent via WebSocket containing a type (e.g., public_upload, join_group) and a payload.
    - *Image Data:* Images are converted to Base64 strings to be embedded within the JSON payload.
- **Output:**
    - *Server Response:* A JSON object broadcasted to relevant clients containing the type (e.g., receive_public, group_notification) and the payload (image data, username, comment).
    - *Visual:* The client renders the Base64 string into an <img> tag and appends it to the DOM.

---

2. Implementation

2.1 Tech Stack

- **Runtime:** Node.js
- **Frameworks:** Express (Web Framework), ws (WebSocket implementation).
- **Frontend:** Vanilla JavaScript, HTML5, CSS3.

2.2 Server-Side Logic (server.js)

The core logic resides in the wss.on("connection") event listener. The server handles concurrency using Node.js's event-driven architecture. Messages are processed via a switch statement handling three main operation types:

1. **join_group**: Updates the clientRooms map to associate a specific WebSocket connection with a room name. It notifies the user and the room members.
2. **public_upload**: Calls broadcastToAll, iterating through wss.clients to send data to every open connection.
3. **group_upload**: Calls broadcastToRoom, which filters clients based on the clientRooms map before sending data.

Code Snippet: Room Broadcasting Logic

JavaScript

```
function broadcastToRoom(room, data) {
  wss.clients.forEach((client) => {
    // Check if client is in the specific room map
    const rooms = clientRooms.get(client);
    if (client.readyState === WebSocket.OPEN && rooms && rooms.has(room)) {
      client.send(JSON.stringify(data));
    }
  });
}
```

2.3 Client-Side Logic (script.js)

The client manages the connection state and formats data for the server. Crucially, it handles binary image data by converting it to text-based Base64 format to ensure it can be serialized

into JSON.

Code Snippet: Base64 Conversion

JavaScript

```
function getBase64(file) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file); // Converts file to Data URL (Base64)
    reader.onload = () => resolve(reader.result);
    reader.onerror = (error) => reject(error);
  });
}
```

---

3. Test Cases

To verify the correctness of the program, the following test cases were executed.

Test Case	Action	Input Data	Expected Outcome	Status
1. Public Sharing	User A uploads a photo to the Public Feed.	File: cat.jpg, Comment: "Hello"	User B (on a different browser) sees the photo and comment immediately in the Public Feed.	**PASS**
2. Group Joining	User A enters a room name and clicks "Join".	Room: "CS-Project"	System displays "You joined group: CS-Project" and notifies other room members.	**PASS**

3. Group Isolation	User A is in "Room1". User B is in "Room2". User A uploads a photo.	File: diagram.png	User B does **not** receive the photo. User C (also in "Room1") **does** receive it.	**PASS**
4. Large Payload	User uploads a high-resolution image.	Image > 5MB	Server accepts the payload (Max configured to 100MB) and broadcasts it.	**PASS**
5. Disconnection	User A closes the browser tab.	N/A	Users in the same group as User A receive a "User has left the group" notification.	**PASS**

---

4. Results

Performance Metrics:

- **Latency:** Messages appear near-instantaneously (< 100ms on localhost) because the WebSocket connection remains open, eliminating the TCP handshake overhead required for standard HTTP requests.
- **Concurrency:** The server successfully handled multiple client windows simultaneously. The wss.clients.forEach loop efficiently distributed messages to all connected clients.

Observation:

When a user joins a group, the clientRooms Map successfully updates. The application distinguishes clearly between public_upload (broadcast) and group_upload (multicast), verifying the protocol layer logic.

---

5. Analysis

Discussion on Protocol Layers:

The application effectively layers a custom JSON-based application protocol over the WebSocket transport layer.

- **Concurrency Handling:** Node.js handles concurrent connections using a single-threaded event loop. This is efficient for I/O-bound tasks like message passing. However, heavy computation (e.g., image processing) could block the loop.

- **Data Overhead:** Converting images to Base64 increases the data size by approximately 33% compared to binary transfer. While simple to implement, this is less bandwidth-efficient than sending binary blobs directly.

Known Limitations & Improvements:

1. **Persistence:** The application stores room data in memory (Map). If the server restarts, all room assignments are lost. *Improvement: Use Redis or a database*.
2. **Scalability:** Broadcasting relies on iterating through all clients in memory. For a distributed system, a Pub/Sub mechanism (like Redis Pub/Sub) would be required to sync across multiple server instances.
3. **Security:** There is no authentication. Any user can pick any username. *Improvement: Implement JWT authentication*.

---

6. Comments

Evaluation:

This assignment provided valuable insight into full-duplex communication protocols. It demonstrated how WebSockets solve the "polling" problem of HTTP by allowing the server to push data to the client.

Learning Outcomes:

- Learned how to upgrade an HTTP server to a WebSocket server using the ws library.
- Understood the mechanism of "rooms" or "channels" by logically grouping socket connections in server-side data structures.
- Gained experience in handling binary data (images) within a text-based JSON protocol.

Difficulty:

The assignment was of moderate difficulty. The most challenging aspect was managing the state of users (which rooms they are in) and ensuring that "leave" notifications triggered correctly upon socket disconnection.